Integración entre equipos — Tienda Virtual

De: Equipo B (Catálogo y Host App) · **Para:** Equipo A (Búsqueda) y Equipo C (Carrito)
Fecha: 5 de noviembre de 2026
Plazo para objetar: 8 de noviembre de 2026. Si no hay objeciones antes de esa fecha, lo de este documento queda acordado.

Este documento reúne los dos contratos que rigen la integración:

1. **Contrato de Catálogo v2.4:** qué ofrece Catálogo por REST y por eventos, y cómo lo usa cada equipo.
2. **Contrato del Host App v1.2:** cómo se carga cada módulo en el cascarón y cómo se pasa de uno a otro.

Todos los cambios son **compatibles** con lo que ya funciona: nadie pierde nada de lo acordado antes.

Qué le toca a cada equipo

Equipo	Qué ajusta	Dónde está
A — Búsqueda	Enlazar su cola al exchange `catalogo.eventos` con la routing key `producto.*` (hoy escucha `catalogo.events` con `catalogo.producto.*`, y no le llega ningún evento)	Catálogo, 6.1 y 6.4
A — Búsqueda	Aplicar `producto.reactivado`, y no marcar un producto como activo por su cuenta: usar el campo `activo` del evento	Catálogo, 5 y 6.1
A — Búsqueda	Apuntar `RABBITMQ_URL` al RabbitMQ de Catálogo	Catálogo, 10.1
A — Búsqueda	Frontend en el puerto `5174`, llamando a su backend por Kong, y sin su encabezado cuando está dentro del Host App	Host App, 4 y 9.3
C — Carrito	Nada en el backend: ya envía la `referencia` y ya llama a `devolver-stock`	Catálogo, 7
C — Carrito	Frontend en el puerto `5175`; al cargar, si la dirección trae `agregar`, agregar ese producto; y sin su encabezado cuando está dentro del Host App	Host App, 4, 9.1 y 9.3
B — Catálogo	Implementar la `referencia` idempotente, `devolver-stock`, `items` en el 409, y `activo` y `fecha` en los eventos	Catálogo, 2, 4, 5 y 11
B — Catálogo	Un solo Kong con las rutas de los tres, y un `docker-compose` de integración en el repositorio del Host App	Catálogo, 10
B — Host App	El botón «Agregar al carro» y el mensaje `navegar`	Host App, 4 y 9.1

Lo que se decidió

Tema	Decisión
RabbitMQ	Uno solo para todos: el de Catálogo

Tema	Decisión
Kong	Uno solo con las rutas `/api/catalogo`, `/api/busqueda` y `/api/carrito`, operado por el Equipo B
Integración y demostración	Un `docker-compose` en el repositorio del Host App, que construye los tres backends desde sus repositorios. Hacia afuera solo se publican Kong (`8000`) y la consola de RabbitMQ (`15672`)
Puertos de los frontends	Catálogo `5173`, Búsqueda `5174`, Carrito `5175`, Host App `5050`
Botón «Agregar al carro»	Catálogo le pide al Host App navegar a `/carrito?agregar={id}`, y Carrito agrega el producto al cargar
Descuento de stock	Idempotente con `referencia` (Carrito usa su `checkoutId`), y se devuelve con `devolver-stock`. No vence solo

Para objetar o preguntar: respondan antes del 8 de noviembre de 2026 a Rances Ramírez (Equipo B), indicando la sección.

Versión en línea de los contratos (mientras corre el plazo están en su propia rama; al quedar acordados pasan a `main`):

- Catálogo v2.4: https://github.com/rancesra/teambsoft-backend/blob/contrato-v2.4/docs/CONTRATO-CATALOGO.md
- Host App v1.2: https://github.com/rancesra/teambsoft-hostapp/blob/contrato-v1.2/CONTRATO-HOSTAPP.md

Contrato de Servicio — Catálogo (Equipo B)

Versión: 2.4 — Segunda entrega (Ciclo 2)
Fecha: 2026-11-05
Equipo responsable: Equipo B
Consumido por: Equipo A (Búsqueda), Equipo C (Carrito)
Estado: propuesta, con plazo para objetar hasta el **8 de noviembre de 2026** (sección 11). Si los equipos A y C no objetan antes de esa fecha, queda acordada. Todos los cambios son compatibles con la v2.3.

1. Propósito

Catálogo es la fuente de verdad de los productos de la tienda. Expone operaciones CRUD vía REST (a través de Kong) y publica eventos a RabbitMQ para que otros servicios se mantengan sincronizados sin tener que consultarlo en cada operación.

2. Endpoints REST

Se consumen a través del API Gateway (Kong), con el prefijo `/api/catalogo`. Kong lo quita antes de llegar al servicio (`strip_path`): `/api/catalogo/productos` llega a Catálogo como `/productos`.

Acceso	URL base	Cuándo se usa
A través de Kong	`http://localhost:8000/api/catalogo`	El navegador y la integración final
Directo al servicio	`http://localhost:8080` (sin prefijo)	Pruebas de cada equipo en su propia máquina, sin Kong
Dentro de la red de integración	`http://kong:8000/api/catalogo` o, directo, `http://catalogo:8080`	Servicio a servicio, en el `docker-compose` de integración (sección 10)

Método	Endpoint	Descripción	Auth
GET	/productos	Lista productos activos, paginado, filtro opcional `?categoria=`	Pública
GET	/productos/{id}	Detalle de un producto	Pública
GET	/categorias	Lista las categorías válidas	Pública
POST	/productos	Crea un producto	Admin
PUT	/productos/{id}	Actualiza un producto	Admin
DELETE	/productos/{id}	Desactiva un producto (soft delete)	Admin
POST	/productos/{id}/activar	Reactiva un producto desactivado	Admin
POST	/productos/descontar-stock	Descuenta stock al confirmar una compra	Servicio
POST	/productos/devolver-stock	Devuelve el stock de un descuento hecho con `referencia`	Servicio

> Identity Server (Keycloak) no está integrado en esta entrega. Los endpoints Admin no validan token real todavía — se documenta el contrato final para que Carro y Búsqueda ya sepan qué esperar cuando se conecte.

GET /productos → 200

```
{
  "productos": [ { "...": "ver modelo Producto en sección 3" } ],
  "total": 42,
  "pagina": 1,
  "tamanoPagina": 20
}
```

Parámetros de consulta (todos opcionales, se pueden combinar):

Parámetro	Por defecto	Reglas
`categoria`	todas	`id` de una categoría (ver `GET /categorias`)
`pagina`	1	entero ≥ 1 — la primera página es la 1
`tamanoPagina`	20	entero entre 1 y 100
`activo`	`true`	`true` (solo activos) · `false` (solo desactivados) · `todos`

Ejemplo: `GET /productos?categoria=cat-ropa&pagina=2&tamanoPagina=20`

- `total` es la cantidad de productos que cumplen el filtro, no solo los de la página actual.
- **`activo` existe para la pantalla de administración**, que necesita poder encontrar lo que desactivó. Su valor por defecto es `true`, así que quien no lo envíe sigue recibiendo solo productos activos, exactamente como antes. El escaparate público no debe enviarlo nunca.
- Si `pagina` es mayor que la última página, responde 200 con `productos: []` y el mismo `total`.
- Si `categoria` no corresponde a ninguna categoría existente, responde 200 con `productos: []` y `total: 0` (un filtro sin resultados no es un error).
- Valores de `pagina` o `tamanoPagina` fuera de estas reglas → 400 (`VALIDACION_FALLIDA`).

GET /productos/{id} → 200 (Producto) · 404 (`PRODUCTO_NO_ENCONTRADO`)

Devuelve el producto aunque esté inactivo (viene con `activo: false`). El 404 es solo para un `id` que no existe.

GET /categorias → 200

```
[
  { "id": "cat-ropa", "nombre": "ropa" },
  { "id": "cat-hogar", "nombre": "hogar" },
  { "id": "cat-electronica", "nombre": "electrónica" }
]
```

POST /productos → 201 (Producto creado, con `id` asignado) · 400 (`VALIDACION_FALLIDA`)

PUT /productos/{id} → 200 (Producto actualizado) · 404 · 400

Reemplaza el producto completo: el cuerpo lleva todos los campos, con las mismas reglas que `POST` (sección 3).

- No cambia `id` (se toma de la URL) ni `activo` (solo lo cambia `DELETE`); si vienen en el cuerpo, se ignoran.
- Sobre un producto inactivo responde 200 y el producto sigue inactivo.

DELETE /productos/{id} → 204 (marca `activo:false` , no borra el registro) · 404

Si el producto ya estaba inactivo, responde 204 sin cambiar nada ni publicar otro evento (la operación es idempotente).

POST /productos/{id}/activar → 200 (Producto reactivado) · 404 (`PRODUCTO_NO_ENCONTRADO`)

Vuelve a poner `activo: true` . Es la operación inversa de `DELETE` y no existía hasta v2.3: `PUT` ignora `activo` a propósito, así que sin este endpoint un producto desactivado no tenía camino de vuelta.

- Si el producto ya estaba activo, responde 200 sin cambiar nada ni publicar otro evento (idempotente, igual que `DELETE`).
- No toca ningún otro campo: para cambiar datos está `PUT` .

POST /productos/descontar-stock → 200 · 409 (`STOCK_INSUFICIENTE` , `REFERENCIA_EN_USO`) · 404 · 400

Descuenta stock de varios productos a la vez. Lo llama **Carro al confirmar el checkout**; no lo llama el navegador.

Cuerpo de la petición:

```
{ "referencia": "chk_7f3a9c21", "items": [ { "id": "64f1a2b3c9e77a001f3d8e21", "cantidad": 2 }
] }
```

Campo	Tipo	Reglas
`referencia`	string	**Opcional** (v2.4), de 1 a 100 caracteres. Identifica la compra; Carro usa su `checkoutId`
`items`	array	Requerido, no vacío, sin ids repetidos
`items[].id`	string	Id de un producto existente y activo

Campo	Tipo	Reglas
`items[].cantidad`	integer	Requerido, ≥ 1

Respuesta 200:

```
{ "referencia": "chk_7f3a9c21", "items": [ { "id": "64f1a2b3c9e77a001f3d8e21", "stockRestante": 118 } ] }
```

`referencia` solo viene en la respuesta si vino en la petición.

Reglas:

- Cada descuento se hace con una operación atómica de MongoDB condicionada al stock disponible, así que **dos checkouts simultáneos sobre el mismo producto no pueden pasar los dos**.
- Si algún ítem no alcanza, responde **409** (`STOCK_INSUFICIENTE`) con los productos afectados en `items` (sección 4), y **revierte los descuentos que ya había hecho** en esa misma petición: o pasa todo, o no pasa nada.
- `cantidad` debe ser un entero ≥ 1, y el producto debe existir y estar activo; si no, 400 o 404.
- Publica `producto.actualizado` por cada producto cuyo stock cambió, para que el índice de Búsqueda no se desfase.

Idempotencia por `referencia` (v2.4):

- **Si llega dos veces la misma `referencia`** con los mismos ítems, la segunda responde **200 con el mismo cuerpo que la primera**, sin descontar otra vez ni publicar eventos. Así, quien reintenta porque no recibió la respuesta no descuenta dos veces.
- **Si llega la misma `referencia` con otros ítems o cantidades**, responde **409** **`REFERENCIA_EN_USO`** y no descuenta nada: una referencia identifica una sola compra.
- **Un descuento que falla** (409 `STOCK_INSUFICIENTE` , 404 o 400) **no registra la referencia**, así que se puede reintentar con la misma.
- **Sin `referencia`** , el endpoint funciona exactamente como en la v2.3, sin protección contra reintentos.

Lo que este endpoint no es: una reserva con retención. No aparta stock mientras el usuario decide y el descuento no vence solo: es definitivo hasta que se devuelve con `devolver-stock` . Ver sección 8.

POST /productos/devolver-stock → 204 · 400 (v2.4)

Revierte un descuento hecho con `referencia` . Lo llama **Carro si falla guardar el pedido** después de descontar.

Cuerpo de la petición:

```
{ "referencia": "chk_7f3a9c21" }
```

Reglas:

- Devuelve a cada producto las unidades que se le descontaron con esa referencia y publica `producto.actualizado` por cada uno.
- **Es idempotente:** si la referencia ya se devolvió, o no existe, responde 204 sin cambiar nada.
- Devuelve el stock aunque el producto se haya desactivado entretanto: las unidades existen igual.
- Sin `referencia`, o con una vacía, responde 400 (`VALIDACION_FALLIDA`).
- Una referencia devuelta queda cerrada: si después llega un `descontar-stock` con esa misma referencia, responde 409 `REFERENCIA_EN_USO`. Para comprar de nuevo, Carro usa una referencia nueva.

3. Modelo de Producto

Campo	Tipo	Descripción
id	string	Identificador único (generado por Mongo)
nombre	string	Requerido, máx. 120 caracteres
descripcion	string	Opcional
precio	number	Requerido, > 0
categoria	string	Requerido, debe coincidir con un `id` de `GET /categorias`
stock	integer	Requerido, >= 0
imagenes	array[string]	Opcional, URLs
activo	boolean	`true` al crearse; `false` tras un soft delete. Solo lo cambia `DELETE`

Ejemplo:

```
{
  "id": "64f1a2b3c9e77a001f3d8e21",
  "nombre": "Camiseta básica algodón",
  "descripcion": "Camiseta unisex 100% algodón, corte regular.",
  "precio": 49900,
  "categoria": "cat-ropa",
  "stock": 120,
  "imagenes": ["https://cdn.tienda.com/img/camiseta-1.jpg"],
  "activo": true
}
```

Modelo de Categoría

Campo	Tipo	Descripción
id	string	Identificador único, usado como valor de `Producto.categoria`
nombre	string	Nombre visible de la categoría

En esta entrega las categorías vienen precargadas (seed) — ver sección 8.

4. Formato de error estándar

```
{
  "codigo": "PRODUCTO_NO_ENCONTRADO",
  "mensaje": "descripción legible del error"
}
```

Códigos de esta entrega:

Código	HTTP	Cuándo
`PRODUCTO_NO_ENCONTRADO`	404	El id no existe
`VALIDACION_FALLIDA`	400	La petición no cumple las reglas del contrato
`STOCK_INSUFICIENTE`	409	Algún ítem de `descontar-stock` no alcanza
`REFERENCIA_EN_USO`	409	La `referencia` ya se usó con otros ítems, o ya se devolvió (v2.4)

`STOCK_INSUFICIENTE` es 409 y no 400 porque no es un error de formato de la petición, sino un conflicto con el estado actual del recurso: la misma petición puede fallar ahora y funcionar en un minuto. Se usa el mismo código y el mismo número que ya definió Carro en su contrato v1.1.

Desde la v2.4, `STOCK_INSUFICIENTE` trae `items` con los productos que no alcanzaron y el stock que tenían en ese momento:

```
{
  "codigo": "STOCK_INSUFICIENTE",
  "mensaje": "No hay stock suficiente para 1 producto",
  "items": [ { "id": "64f1a2b3c9e77a001f3d8e21", "cantidad": 3, "stock": 1 } ]
}
```

Un error puede traer campos adicionales, como `items`, pero **nunca reemplazan a `codigo` ni a `mensaje`**: quien solo lea esos dos sigue funcionando igual.

5. Eventos publicados (RabbitMQ)

Catálogo publica en un exchange de RabbitMQ de tipo **topic** llamado `catalogo.eventos`. Cada evento sale con su routing key; cada consumidor crea su propia cola y la enlaza al exchange con las routing keys que le interesan (por ejemplo, `producto.*` para recibirlos todos). El cuerpo del mensaje es un JSON con los campos del payload.

El exchange es **durable** y no se borra solo (`durable: true`, `autoDelete: false`). Quien lo declare de su lado tiene que usar esos mismos argumentos, o RabbitMQ rechaza la declaración.

Evento	Routing key	Se publica cuando	Payload
ProductoCreado	`producto.creado`	se crea un producto	id, nombre, descripcion, precio, categoria, stock, **activo**, **fecha**
ProductoActualizado	`producto.actualizado`	se modifica cualquier campo, incluido el stock (`PUT` , `descontar-stock` , `devolver-stock`)	id, nombre, descripcion, precio, categoria, stock, **activo**, **fecha**
ProductoDesactivado	`producto.desactivado`	se hace soft delete (no se repite si el producto ya estaba inactivo)	id, **fecha**
ProductoReactivado	`producto.reactivado`	se reactiva un producto desactivado (no se repite si ya estaba activo)	id, nombre, descripcion, precio, categoria, stock, **activo**, **fecha**

Los campos en negrita son nuevos en la v2.4:

- **`activo`** (boolean): el estado del producto en el momento del evento. **`producto.actualizado` no implica que el producto esté activo:** un `PUT` sobre un producto desactivado publica `actualizado` y el producto sigue inactivo. Por eso cada evento lleva el estado completo y el consumidor no tiene que suponerlo.
- **`fecha`** (string, ISO-8601 UTC, por ejemplo `"2026-11-05T15:30:00Z"`): el instante en que Catálogo publicó el evento. Sirve para descartar un evento más viejo que el último que ya se aplicó a ese producto.

Ejemplo de `producto.creado`:

```
{
  "id": "64f1a2b3c9e77a001f3d8e21",
  "nombre": "Camiseta básica algodón",
  "descripcion": "Camiseta unisex 100% algodón.",
  "precio": 49900,
  "categoria": "cat-ropa",
  "stock": 120,
  "activo": true,
  "fecha": "2026-11-05T15:30:00Z"
}
```

`producto.desactivado` lleva solo `{ "id": "...", "fecha": "..." }`.

Cuántos consumidores puede haber: los que hagan falta. Al ser un exchange *topic*, cada servicio crea su propia cola y recibe su propia copia de cada mensaje; que se sumen consumidores no afecta a los que ya estaban ni obliga a cambiar nada en Catálogo.

Por qué `ProductoReactivado` sigue siendo un evento aparte, aunque desde la v2.4 el payload trae `activo`: quien solo necesita reaccionar a las reactivaciones se enlaza a esa routing key y no tiene que revisar cada `actualizado`. Lleva el payload completo porque quien había sacado el producto de su índice necesita todos los datos para volver a meterlo.

5.1 Reglas para consumir los eventos

1. **Procesar el mismo evento dos veces no debe romper nada.** Reindexar o actualizar en caché el mismo producto con los mismos datos es inofensivo. Hoy un evento no se repite, pero puede pasar cuando Catálogo implemente el Outbox (evaluación).
2. **Un evento puede llegar después de otro más reciente** del mismo producto. Quien guarde la `fecha` del último evento aplicado a cada producto descarta los más viejos.
3. **Hoy un evento se puede perder** si RabbitMQ está caído en el momento de publicar. Mientras no exista el Outbox, la forma de recuperarse es reconstruir el índice o la caché con `GET /productos?activo=todos`, de forma excepcional y no como mecanismo normal.

6. Integración con Equipo A (Búsqueda)

Búsqueda **no debe** llamar a `GET /productos` para construir su índice. Debe suscribirse a los eventos de la sección 5 y mantener su propio índice de forma asíncrona. La única excepción es reconstruir el índice después de una pérdida de eventos (regla 3 de la sección 5.1).

6.1 Lo que hace Búsqueda

1. Declara su propia cola **durable** (por ejemplo, `busqueda.catalogo.eventos`) y la enlaza al exchange `catalogo.eventos` con la routing key `producto.*`, que recibe los cuatro eventos.
2. Aplica cada evento así:

Evento	Qué hace Búsqueda en su índice
`producto.creado`	Inserta el producto (o lo actualiza si ya existía), con `activo` tal como viene
`producto.actualizado`	Actualiza el producto con todos los campos del payload, **incluido `activo` tal como viene**. Nunca lo marca activo por su cuenta
`producto.desactivado`	Marca el producto como inactivo y deja de mostrarlo. Si no lo tenía, no hace nada
`producto.reactivado`	Inserta o actualiza el producto con el payload completo y vuelve a mostrarlo

3. Si guarda la `fecha` del último evento aplicado a cada producto, descarta los eventos más viejos (sección 5.1).
4. Un evento que no pueda procesar va a su cola de mensajes fallidos (*dead-letter*), sin bloquear a los demás.

6.2 Lo que le garantiza Catálogo

- Un evento por cada cambio real: crear, actualizar (incluido el stock), desactivar y reactivar. Las operaciones idempotentes repetidas no publican otra vez.

- El evento sale **después** de guardar en MongoDB: nunca se anuncia un producto que no existe.
- Los campos de la sección 5, en JSON, sin envoltorio: el cuerpo del mensaje es el payload directamente.
- La prueba automática `PublicacionEventosTest` comprueba en cada cambio de código que los eventos salen con la routing key y el cuerpo del contrato.

6.3 Historial de acuerdos con Búsqueda

- **v2.3:** `descripcion` y `stock` en el payload (el contrato de Búsqueda v1.0 lo marcaba como bloqueante para la búsqueda de texto completo) y el evento `producto.reactivado`, que preguntaban.
- **v2.4:** `activo` y `fecha` en el payload, y las reglas de consumo de la sección 5.1.

6.4 Verificación del 5 de noviembre de 2026

Revisado el consumidor de Búsqueda (`src/infrastructure/rabbitmq/catalogoConsumer.js`, rama `main`):

	Contrato	Lo que escucha Búsqueda	Efecto
Exchange	`catalogo.eventos`	`catalogo.events`	No recibe ningún evento
Routing keys	`producto.*`	`catalogo.producto.creado`, `.actualizado`, `.desactivado`	No coincide ninguna
Reactivación	`producto.reactivado`	No la escucha	Un producto reactivado sigue oculto
`producto.actualizado`	Trae `activo`	Marca siempre `activo: true`	Un producto retirado y editado vuelve a aparecer

El exchange y las routing keys son los del contrato desde la v2.1 (10 de septiembre de 2026) y no cambian en la v2.4. Para recibir los eventos, Búsqueda ajusta esos tres puntos en su consumidor.

7. Integración con Equipo C (Carrito)

Carro consume Catálogo por REST, de forma síncrona: necesita el precio, el estado y el stock en el momento, antes de dejar continuar al usuario.

7.1 Qué llama Carro en cada paso

Paso en Carro	Llamada a Catálogo	Qué hace con la respuesta
Agregar un producto al carrito (desde su propia pantalla, o al abrir `/carrito?agregar={id}` desde el botón de Catálogo)	`GET /productos/{id}`	Valida que exista y que `activo == true`, y guarda el `nombre` y el `precio` de Catálogo, **nunca los que mande el frontend**

Paso en Carro	Llamada a Catálogo	Qué hace con la respuesta
Mostrar el carrito y verificar disponibilidad (HU-09)	`GET /productos/{id}` por cada ítem	`404` o `activo: false` → el ítem queda no disponible; `stock` menor que la cantidad → sin stock; precio distinto → avisa el cambio
Confirmar el pedido (HU-10)	`POST /productos/descontar-stock` con `referencia` = `checkoutId`	`200` → guarda el pedido. `409 STOCK_INSUFICIENTE` → muestra los `items` que fallaron; no se descontó nada
Si falla guardar el pedido después de descontar	`POST /productos/devolver-stock` con la misma `referencia`	`204` → el stock volvió

7.2 Reglas

- **Reintentos:** si Carro no recibe respuesta de `descontar-stock` (timeout o error de red), reintenta **con la misma `referencia`**. Si el primer intento sí descontó, el reintento devuelve el mismo resultado sin descontar otra vez.
- **Un producto desactivado** responde `200` con `activo: false`, no `404`. El `404` es solo para un id que no existe. Si se desactiva estando en un carrito, Carro lo marca como no disponible en vez de quitarlo en silencio (contrato de Carrito v1.1, secciones 2.1 y 8.A).
- **Los ids son de MongoDB:** 24 caracteres hexadecimales.
- **La categoría** viene como id en el campo `categoria` (por ejemplo, `cat-ropa`). Los nombres salen de `GET /categorias`.
- **Eventos (opcional):** Carro puede suscribirse a `catalogo.eventos` como un consumidor más (sección 5) y usar `producto.actualizado` y `producto.desactivado` para mantener su caché, reservando la llamada REST para el checkout.

7.3 Historial de acuerdos con Carrito

- **v2.3:** `descontar-stock`, que el contrato de Carrito v1.1 marcaba como bloqueante para su HU-09, y el error `STOCK_INSUFICIENTE`, con el mismo nombre y número que ya usaba Carrito.
- **27 de octubre de 2026:** respuesta a las preguntas del Equipo C (producto inactivo, categoría, stock, token y URL), recogida en la sección 7.1 del contrato de Carrito v1.3.
- **v2.4:** la `referencia` idempotente, `devolver-stock` y `items` en el 409. Es lo que el contrato de Carrito v1.3 (sección 7.2) propuso a Catálogo. El Carrito ya envía la `referencia` y ya llama a `devolver-stock`; mientras Catálogo no lo implemente, la referencia se ignora y la devolución falla, y Carrito la registra en su log para hacerla a mano.

8. Fuera de alcance en esta entrega

- Autenticación real vía Keycloak
- Descuento y Órdenes (equipo aún sin asignar)
- Variantes de producto (talla/color con stock por variante)
- Filtros y ordenamiento avanzado (solo se filtra por categoría y por `activo` en esta entrega)

- **Reserva de stock con retención:** `POST /productos/descontar-stock` descuenta en el momento del checkout, pero no aparta unidades mientras el usuario decide, y **un descuento no vence solo**: hacerlo vencer necesita un proceso que lo libere por tiempo, que nadie opera
- **Reposición automática de stock:** desde la v2.4 existe `devolver-stock`, pero solo se ejecuta si Carro lo llama. Si un pedido se pierde sin que nadie lo devuelva, se corrige a mano
- **Entrega garantizada de los eventos (Transactional Outbox):** evaluado en EVALUACION-OUTBOX.md; se recomienda para una entrega posterior
- **Atomicidad entre varios productos:** MongoDB garantiza la atomicidad por documento, y esta entrega corre un Mongo de un solo nodo, sin transacciones. Cuando un ítem falla, Catálogo revierte los descuentos ya hechos en esa petición, pero esa reversión es de mejor esfuerzo: si el servicio se cae justo en medio, puede quedar diferencia
- Gestión de categorías (`POST` / `PUT` / `DELETE /categorias`) — las categorías vienen precargadas por ahora, solo se listan

9. Frontend — módulo Catálogo

El módulo "Catálogo" del Cliente Web (dentro del Host App) consume estos endpoints directamente, a través de Kong:

Vista	Endpoint que consume	Qué muestra
Listado de productos	`GET /productos`	Grid o lista: nombre, precio, imagen, categoría
Detalle de producto	`GET /productos/{id}`	Vista individual con toda la información
Crear producto (admin)	`POST /productos`	Formulario: nombre, descripción, precio, categoría, stock, imágenes
Selector de categoría	`GET /categorias`	Dropdown para el formulario de creación
Administración de productos	`POST`, `PUT`, `POST /{id}/activar`, `DELETE`	Listado interno, formulario de creación y edición, reactivar y desactivar

Framework: Vue.js, acordado por los 3 equipos para todos los módulos del Host App, de modo que sean consistentes entre sí.

El Host App lo construye el Equipo B. Acordado por los 3 equipos el 2026-09-21. La superficie de integración —espacio de rutas de cada módulo, qué expone cada uno, qué entrega el cascarón y las variables de estilo compartidas— está en un documento aparte: CONTRATO-HOSTAPP.md, en el repositorio teambsoft-hostapp. Ese documento es el que tienen que leer los equipos A y C para que su módulo encaje.

Lo esencial para este contrato:

- El módulo de Catálogo vive bajo el prefijo `/catalogo/*` y no pinta nada fuera de él.
- **Cada módulo tiene que funcionar solo, sin el Host App.** Es lo que evita que un equipo quede bloqueado esperando a otro, y es el plan B si la integración falla el día de la sustentación.

- La integración se hace en dos etapas: primero cada módulo se carga en un iframe, que es obligatoria (contrato del Host App v1.1); encima, Module Federation.

El botón "Agregar al carro" (v2.4) funciona así:

1. Catálogo lo pinta en la vista de detalle **solo si el producto está activo, tiene stock y el módulo está dentro del Host App**. Si corre solo, no hay carrito al cual llevar al usuario, así que el botón no aparece.
2. Al pulsarlo, Catálogo le pide al Host App que navegue a `/carrito?agregar={id}`, con el mensaje `navegar` (contrato del Host App, sección 4).
3. El módulo de Carrito abre con esa dirección, agrega una unidad del producto con su propio endpoint y muestra el carrito.

Así, cada módulo sigue llamando solo a su propio backend, y Catálogo no necesita el token de Carrito. El detalle, con las opciones descartadas, está en la sección 9.1 del contrato del Host App v1.2.

No necesita ser un diseño elaborado: con que consuma los endpoints reales (nada de datos mockeados) y refleje los estados ya definidos (producto inactivo, error 404, etc.) alcanza para el nivel de esta entrega.

10. Infraestructura compartida

Revisados el 5 de noviembre de 2026 los `docker-compose.yml` de los tres equipos, cada uno levanta su propio RabbitMQ o su propio Kong, y varios usan los mismos puertos. Así, los eventos de Catálogo no le llegan a Búsqueda y los servicios chocan si corren en la misma máquina. La v2.4 lo resuelve así:

10.1 Un solo RabbitMQ: el de Catálogo

- Es el `rabbitmq:4-management` del `docker-compose.yml` de Catálogo, con healthcheck y credenciales en variables de entorno (`RABBITMQ_USER` y `RABBITMQ_PASSWORD`).
- Se eligió el de Catálogo porque es quien publica, y porque su prueba automática (`PublicacionEventosTest`) ya comprueba los eventos contra él.
- **Búsqueda** apunta su `RABBITMQ_URL` a ese RabbitMQ: `amqp://<usuario>:<clave>@rabbitmq:5672` dentro de la red de integración, o `@localhost:5672` en una máquina de desarrollo. Las credenciales se comparten por un canal privado, nunca en el repositorio.
- RabbitMQ 4 no acepta colas no durables que no sean exclusivas: la cola de Búsqueda tiene que ser durable, como ya lo es.

10.2 Un solo Kong, con las rutas de los tres

Ruta	Servicio	`strip_path`	Por qué
`/api/catalogo`	`catalogo:8080`	`true`	Los controllers de Catálogo no llevan el prefijo
`/api/busqueda`	`busqueda:3000`	`false`	Las rutas de Búsqueda ya empiezan por `/api/busqueda`
`/api/carrito`	`carrito:3000`	`false`	Las rutas de Carrito ya empiezan por `/api/carrito`

La configuración es declarativa (`kong.yml`, sin base de datos) y la mantiene el Equipo B.

10.3 Un `docker-compose` de integración, en el repositorio del Host App

- Vive en teambsoft-hostapp y lo opera el **Equipo B**. Levanta los tres backends, sus bases de datos, el RabbitMQ de 10.1 y el Kong de 10.2.
- **Construye cada backend desde su repositorio de GitHub**, con el `Dockerfile` de cada equipo: nadie copia código de otro.
- **Dentro de la red de Docker** cada servicio usa sus puertos de siempre sin chocar (Carrito y Búsqueda pueden seguir los dos en el 3000). **Hacia afuera solo se publican Kong (`8000`) y la consola de RabbitMQ (`15672`).**
- Cada equipo sigue teniendo su propio `docker-compose.yml` para desarrollar solo. El de integración es para probar juntos y para la demostración.

10.4 Puertos de los frontends

Módulo	Puerto	Llama a su backend a través de
Catálogo	`5173`	Kong: `/api/catalogo`
Búsqueda	`5174`	Kong: `/api/busqueda` (su `VITE_API_URL` = `http://localhost:8000/api/busqueda`)
Carrito	`5175`	Kong: `/api/carrito`
Host App	`5050`	No llama a ningún backend

Cada frontend fija su puerto con `server.port` y `server.strictPort: true` en `vite.config.js` , para que Vite no lo cambie solo si está ocupado. El Host App los carga con esas direcciones (contrato del Host App, sección 4).

11. Aprobación de la v2.4

Los equipos A y C tienen hasta el **8 de noviembre de 2026** para objetar cualquier punto. Una objeción se discute y, si cambia algo, se publica en una versión nueva. **Si no hay objeciones antes de esa fecha, la v2.4 queda acordada** y se implementa en Catálogo: primero el contrato, después el código.

Equipo	Respuesta	Fecha
B — Catálogo	*Pendiente* (aprueba el Scrum Master del sprint)	
A — Búsqueda	*Pendiente*	
C — Carrito	*Pendiente*	

Lo que le toca hacer a cada equipo para integrar:

Equipo	Qué ajusta	Sección
A — Búsqueda	Enlazar su cola a `catalogo.eventos` con `producto.*` , aplicar `producto.reactivado` y no marcar activo un producto por su cuenta	6.1 y 6.4

Equipo	Qué ajusta	Sección
A — Búsqueda	Apuntar `RABBITMQ_URL` al RabbitMQ de Catálogo y fijar su frontend en el `5174`	10.1 y 10.4
C — Carrito	Fijar su frontend en el `5175`. Ya envía la `referencia` y ya llama a `devolver-stock`	7 y 10.4
B — Catálogo	Implementar los cambios de la v2.4, el Kong de 10.2 y el `docker-compose` de integración de 10.3	2, 4, 5 y 10

Cambio de la v2.4	Implementado en Catálogo
`referencia` idempotente en `descontar-stock`	Pendiente
`POST /productos/devolver-stock`	Pendiente
`items` en el 409 `STOCK_INSUFICIENTE` y el error `REFERENCIA_EN_USO`	Pendiente
`activo` y `fecha` en los eventos	Pendiente

12. Historial de cambios

Fecha	Cambio
2026-08-20	v1.0 — versión inicial para reunión con equipos A y C
2026-08-20	v1.1 — se agrega `GET /categorias` y modelo de Categoría
2026-08-21	v2.0 — se reasigna a Segunda entrega (Ciclo 2); se agrega sección de Frontend
2026-09-10	v2.1 — Aclaraciones compatibles con v2.0: parámetros de paginación de `GET /productos` (`pagina`, `tamanoPagina`); `?categoria=` inexistente devuelve lista vacía; `GET /productos/{id}` devuelve inactivos con `activo:false`; reglas de `PUT` (reemplazo completo, no cambia `id` ni `activo`) y de `DELETE` (idempotente); exchange `catalogo.eventos` y routing keys de los eventos; `stock` en el formulario de creación; variantes fuera de alcance
2026-09-10	v2.2 — §9: el frontend se construye con Vue.js, acordado por los 3 equipos; sigue pendiente el mecanismo de integración con el Host App
2026-09-21	v2.3 — Cambios compatibles con v2.2, a partir de la revisión cruzada con los contratos de Búsqueda v1.0 y Carrito v1.1. **Eventos:** se agregan `descripcion` y `stock` al payload (lo pedía Búsqueda como bloqueante) y se suma `producto.reactivado`. **Endpoints nuevos:** `POST /productos/{id}/activar` (no existía forma de reactivar) y `POST /productos/descontar-stock` (lo pedía Carrito como bloqueante para su checkout). **`GET /productos`:** nuevo parámetro `activo`, con `true` por defecto, para que la administración pueda encontrar lo que desactivó. **Errores:** se agrega `STOCK_INSUFICIENTE` (409), con el mismo nombre y código que ya usaba Carrito. **§9:** el Host App queda a cargo del Equipo B y su superficie de integración se documenta en `CONTRATO-HOSTAPP.md`. Se cierran los dos “pendiente de confirmar” de las secciones 6 y 7

Fecha	Cambio
2026-11-05	v2.4 — Cambios compatibles con v2.3, a partir de la revisión de los repositorios de Búsqueda y Carrito. **Carrito:** `referencia` opcional e idempotente en `descontar-stock`, nuevo `POST /productos/devolver-stock`, `items` en el 409 `STOCK_INSUFICIENTE` y nuevo error `REFERENCIA_EN_USO` (lo propuso el contrato de Carrito v1.3); la sección 7 detalla qué llama Carro en cada paso. **Búsqueda:** `activo` y `fecha` en el payload de los eventos, reglas de consumo (sección 5.1) y la sección 6 detalla cómo aplicar cada evento. **Sección 2:** URL base a través de Kong y directa. **Sección 9:** la etapa 1 del Host App es con iframes, como dice su contrato v1.1, y se decide el botón “Agregar al carro” (`/carrito?agregar={id}`). **Nuevas secciones:** 10 (infraestructura compartida: un solo RabbitMQ, el de Catálogo; un solo Kong con las rutas de los tres; un `docker-compose` de integración en el repositorio del Host App, operado por el Equipo B; y los puertos de los frontends) y 11 (aprobación, con plazo para objetar hasta el 8 de noviembre)

Contrato del Host App — Cliente Web

Versión: 1.2
Fecha: 2026-11-05
Estado: los cambios de la v1.2 tienen plazo para objetar hasta el **8 de noviembre de 2026**. Si los equipos A y C no objetan antes de esa fecha, quedan acordados.
Equipo responsable: Equipo B (Catálogo)
Consumido por: Equipo A (Búsqueda), Equipo C (Carrito)

Este documento es al frontend lo que los contratos de servicio son al backend: define **la superficie de integración** entre el cascarón y cada módulo. No dice cómo debe estar hecho cada módulo por dentro — eso es asunto de cada equipo.

1. Qué es el Host App y qué no es

El Host App es la aplicación que el usuario abre en el navegador. Es **un cascarón**: encabezado, navegación, enrutador y los espacios donde se montan los módulos.

El Host App sí hace	El Host App no hace
Encabezado, menú y disposición general	Pantallas de negocio
Decidir qué módulo atiende cada ruta	Llamar a ningún microservicio
Definir las variables de estilo compartidas	Guardar datos de negocio
Entregar la sesión a los módulos	Validar reglas de negocio

Si el Host App llama a un endpoint de Catálogo, Búsqueda o Carrito, algo se hizo mal: esa llamada le toca al módulo correspondiente.

2. Espacio de rutas

Cada módulo es dueño de un prefijo y **no debe pintar nada fuera de él**.

Prefijo	Módulo	Equipo
`/catalogo/*`	Catálogo	B
`/busqueda/*`	Búsqueda	A
`/carrito/*`	Carrito	C
`/`	Redirige a `/catalogo`	Host App

Las rutas que cada módulo declara son **relativas a su prefijo**: el módulo declara `''` y `':id'`, y el Host App las monta bajo `/catalogo`. Así, si mañana el prefijo cambia, ningún módulo se entera.

3. La regla de oro

> **Cada módulo tiene que funcionar solo, sin el Host App.**

`npm run dev` en el repositorio de un módulo debe abrir ese módulo completo y usable. No es un capricho:

- Nadie queda bloqueado esperando a otro equipo.
- Cada uno prueba lo suyo de verdad, contra sus endpoints reales.
- Si el día de la sustentación la integración falla, hay plan B: se muestran los tres módulos por separado y se explica el mecanismo. Es mucho mejor que quedarse sin nada.

En la práctica, esto significa que el módulo **no puede asumir que la sesión, la navegación del cascarón o los estilos globales existen**. Ver sección 5.

4. Cómo se integra cada módulo

Se hace en dos etapas. La etapa 1 tiene que estar funcionando antes de intentar la 2.

Etapa 1 — Composición con iframes (obligatoria)

El encabezado del Host App queda fijo, y **cada módulo se carga dentro de un iframe** debajo de él. Cada módulo se construye, se sirve y se despliega por su cuenta, en su propia dirección; el Host App solo sabe dónde está.

Se eligió así porque es la única forma de tener un cascarón de verdad **sin que ningún equipo tenga que cambiar su configuración de build** ni alinear versiones. Y aísla los estilos sin esfuerzo: lo que un módulo declare en CSS no puede pisar a otro.

Lo que tiene que cumplir cada módulo, y nada más:

1. **Estar corriendo en una dirección fija.** El Host App la lee de su archivo `.env.development` (`VITE_MODULO_CATALOGO`, `VITE_MODULO_BUSQUEDA`, `VITE_MODULO_CARRO`). Ese archivo sí se sube al repositorio: son direcciones de desarrollo, no secretos. Si alguien necesita otras en su máquina, las pone en un `.env.development.local`, que git ignora. Cada módulo usa su puerto, con `server.port` y `server.strictPort: true` en su `vite.config.js`, para que Vite no lo cambie solo si está ocupado:

Módulo	Puerto	Dirección
Catálogo	`5173`	`http://localhost:5173`
Búsqueda	`5174`	`http://localhost:5174`
Carrito	`5175`	`http://localhost:5175`
Host App	`5050`	`http://localhost:5050`

   Cada módulo llama a su backend a través del Kong compartido, en `http://localhost:8000` (contrato de Catálogo v2.4, sección 10).

2. **Conservar su prefijo en sus propias rutas.** El Host App le pasa la ruta tal cual: `/carrito/checkout` en el cascarón abre `/carrito/checkout` dentro del módulo de Carro.

3. **No pintar su propio encabezado cuando está dentro del Host App.** Si no, se ven dos. Un módulo sabe que está dentro de un iframe así:

   ```
   const embebido = window.self !== window.top
   ```

   y oculta su barra con `v-if="!embebido"`. Cuando corre solo, la sigue mostrando: la regla de oro se mantiene.

4. **Dejarse cargar en un iframe.** No enviar la cabecera `X-Frame-Options: DENY` ni una política `frame-ancestors 'none'`. Vite no envía ninguna de las dos por defecto.

Si un módulo no tiene dirección configurada, el Host App muestra "*todavía no está conectado*"; si la tiene pero no responde, muestra "*no está disponible*" con un botón para reintentar. Nunca una pantalla en blanco.

Ir de un módulo a otro: el mensaje `navegar`

Un módulo corre dentro de un iframe, en otro puerto, así que no puede cambiar la ruta del cascarón por sí mismo. Cuando necesita llevar al usuario a otro módulo, **se lo pide al Host App** con un mensaje:

```
// Dentro del módulo, solo si está embebido
window.parent.postMessage({ tipo: 'navegar', ruta: '/carrito?agregar=64f1a2b3c9e77a001f3d8e21'
}, '*')
```

El Host App:

1. Solo acepta el mensaje si viene de la dirección de uno de los tres módulos de `.env.development`. Cualquier otro origen se ignora.
2. Solo acepta rutas que empiecen por `/catalogo`, `/busqueda` o `/carrito`.
3. Navega a esa ruta, y el módulo que la atiende se carga con ella, incluida la parte después del `?`.

Así los módulos no se hablan entre sí (sección 8): solo le piden al cascarón que navegue. Es el mismo papel que cumple `navegar` en la etapa 2 (sección 5), pero por mensaje, porque en la etapa 1 no hay `inject`.

Cuando el módulo corre solo, sin el Host App, no hay a quién pedirle que navegue: el módulo decide qué hacer (por ejemplo, ocultar el botón).

Lo que esta etapa no da, y por eso existe la etapa 2:

- **La barra de búsqueda y el contador del carrito no van en el encabezado del cascarón.** Viven dentro de sus módulos. Ponerlos arriba, compartidos, es justamente lo que permite `./Encabezado` en la etapa 2.

- **Si el usuario navega dentro del módulo**, la dirección del cascarón no cambia. Entrar directo a `/catalogo/admin` sí funciona; lo que no se actualiza es la barra de direcciones cuando se navega adentro.

Etapa 2 — Module Federation

El Host App carga los módulos en caliente y comparte con ellos Vue y el enrutador. Cada módulo debe exponer **dos cosas**:

Qué expone	Nombre	Qué es
Sus rutas	`./rutas`	Exportación por defecto: un arreglo de rutas de Vue Router, **relativas al prefijo**
Su pieza del encabezado (opcional)	`./Encabezado`	Un componente de Vue que el Host App monta en el encabezado. Búsqueda pondría ahí su barra; Carrito, su contador

Configuración del lado del módulo:

```
// vite.config.js del módulo
import federation from '@originjs/vite-plugin-federation'

federation({
  name: 'catalogo',                      // 'busqueda' / 'carrito'
  filename: 'remoteEntry.js',
  exposes: {
    './rutas': './src/rutas.js',
    './Encabezado': './src/Encabezado.vue' // solo si el módulo aporta algo al encabezado
  },
  shared: ['vue', 'vue-router']
})
```

Y `src/rutas.js` exporta algo con esta forma:

```
export default [
  { path: '',      name: 'catalogo-listado', component: () => import('./vistas/Listado.vue') },
  { path: ':id',   name: 'catalogo-detalle', component: () => import('./vistas/Detalle.vue') },
]
```

`shared` es obligatorio para `vue` y `vue-router`: si cada módulo trae su propia copia de Vue, la aplicación falla en tiempo de ejecución con errores que no dicen nada útil.

5. Qué le entrega el Host App a los módulos

> **Esto aplica a la etapa 2.** `provide` / `inject` solo funciona cuando todos los módulos viven dentro de la misma aplicación de Vue. En la etapa 1 cada módulo es una aplicación aparte dentro de su iframe, así que **el Host App no les entrega nada**: cada módulo maneja su propia sesión. Como Keycloak no está integrado, hoy eso no le quita nada a nadie.

Se entrega con `provide` / `inject` de Vue. **El módulo debe funcionar si no está** (regla de oro): si `inject` devuelve `undefined`, es que el módulo corre solo.

Clave	Qué es	Si no está
`sesion`	`{ token, userId }` del usuario, o `null` si no ha iniciado sesión	El módulo asume que no hay sesión
`navegar`	Función para ir a otra ruta, incluso de otro módulo	El módulo usa su propio enrutador

Ejemplo dentro de un componente del módulo:

```
const sesion = inject('sesion', null)   // el segundo argumento es el valor por defecto
```

El Host App **no** entrega estado de negocio: ni el carrito, ni los productos, ni los resultados de búsqueda. Si un módulo necesita un dato de otro, lo pide al microservicio que corresponda a través de Kong.

6. Estilo compartido

El Host App define estas variables en `:root`. **Los módulos las usan con `var(--...)` y no escriben colores a mano**, para que las tres partes se vean como la misma tienda.

Variable	Valor	Para qué
`--acento`	`#0F766E`	Botones principales, elementos activos
`--texto`	`#1C1917`	Texto normal
`--texto-suave`	`#57534E`	Texto secundario
`--borde`	`#E7E5E4`	Bordes y separadores
`--fondo`	`#FAFAF9`	Fondo de la página
`--superficie`	`#FFFFFF`	Tarjetas, paneles
`--ok`	`#15803D`	Éxito, disponible
`--aviso`	`#B45309`	Advertencias
`--error`	`#B91C1C`	Errores
`--radio`	`8px` / `12px`	Botones y campos / tarjetas
`--sombra`	`0 4px 14px rgba(28,25,23,.08)`	Elevación de tarjetas

Tipografía: la del sistema (`ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, sans-serif`). No se descarga ninguna fuente.

Aislamiento: cada módulo usa `<style scoped>` en sus componentes. Las clases globales que no se puedan evitar van con el prefijo del módulo (`catalogo-`, `busqueda-`, `carrito-`). Sin esto, los estilos de un módulo se filtran a los otros.

7. Versiones alineadas

Los tres módulos y el Host App tienen que usar la **misma versión mayor y menor de Vue**. La fija el Host App en su `package.json` cuando se cree, y se anuncia en el grupo. Cambiarla es un acuerdo entre los tres equipos, no una decisión de un módulo.

Node.js en versión LTS.

8. Fuera de alcance en esta entrega

- **Autenticación real.** Keycloak no está integrado, igual que en los tres contratos de servicio. En la etapa 1 cada módulo maneja su propia sesión; el contrato de Carro ya prevé un token simulado para sus pruebas.
- **Estado global compartido entre módulos** más allá de `sesion`.
- **Despliegue del Host App** en un servidor: por ahora corre local.
- **Comunicación directa entre módulos.** Si Catálogo necesita avisarle algo al Carrito, se hace por navegación (el mensaje `navegar` de la sección 4, que va al Host App y no al otro módulo) o por el backend. Un módulo nunca le envía mensajes a otro.

9. Decisiones de la v1.2

9.1 El botón “Agregar al carro”

Paso	Quién	Qué hace
1	Catálogo	En la vista de detalle pinta el botón, **solo si el producto está activo, tiene stock y el módulo está embebido**
2	Catálogo	Al pulsarlo, envía `navegar` con la ruta `/carrito?agregar={id}` (sección 4)
3	Host App	Valida el mensaje y navega a `/carrito?agregar={id}`
4	Carrito	Al cargar, si la dirección trae `agregar`, agrega **una unidad** de ese producto con su propio endpoint (`POST /api/carrito/items`) y muestra el carrito
5	Carrito	Quita `agregar` de la dirección (`history.replaceState`), para que recargar la página no lo agregue otra vez

Por qué así: cada módulo sigue llamando solo a su propio backend, y Catálogo no necesita el token de Carrito, que vive en otro origen. Se descartaron dos opciones:

- **Que Catálogo llame directo a la API de Carrito:** rompe la propiedad de cada módulo sobre su servicio, y necesitaría un token que Catálogo no tiene.
- **Que el Host App monte ahí un componente de Carrito:** necesita Module Federation, que es la etapa 2. Cuando llegue, se puede revisar.

Si Carrito no puede agregar el producto (sin sesión, producto inactivo o sin stock), lo resuelve en su propia pantalla, con sus propios mensajes.

9.2 Qué se muestra en `/`

Sigue redirigiendo a `/catalogo`. Una portada propia queda fuera de alcance en esta entrega.

9.3 Lo que le toca a cada equipo

Equipo	Qué ajusta
A — Búsqueda	Fijar su frontend en el `5174` y llamar a su backend por Kong (`VITE_API_URL=http://localhost:8000/api/busqueda`). Ocultar su encabezado cuando está embebido (requisito 3)
C — Carrito	Fijar su frontend en el `5175`, leer `agregar` al cargar (9.1) y ocultar su encabezado cuando está embebido (requisito 3)
B — Catálogo y Host App	El botón y el mensaje `navegar` en Catálogo; recibir y validar `navegar` en el Host App; las direcciones de los tres módulos en `.env.development`

10. Historial de cambios

Fecha	Cambio
2026-09-21	v1.0 — versión inicial. El Host App queda a cargo del Equipo B por acuerdo de los 3 equipos
2026-10-04	v1.1 — **La etapa 1 pasa de enlaces a iframes.** En la v1.0 el Host App solo enlazaba a cada módulo, y al pulsar uno se salía del cascarón. Con iframes el encabezado queda fijo y el módulo se carga adentro, sin que ningún equipo cambie su configuración. Se definen los cuatro requisitos que cumple cada módulo y qué muestra el Host App si uno falta o está caído. Se aclara que `provide` / `inject` (sección 5) solo funciona en la etapa 2: en la v1.0 se prometía para todo, y con enlaces o iframes no se puede. Se quita el requisito de `base` en Vite, que con iframes ya no hace falta. La etapa 2 no cambia
2026-11-05	v1.2 — **Puertos fijos** de los tres módulos y del Host App (requisito 1), y los módulos llaman a su backend por el Kong compartido. **Nuevo mensaje `navegar`** (sección 4) para que un módulo le pida al Host App ir a otro módulo. **Se decide el botón “Agregar al carro”** (sección 9.1): Catálogo navega a `/carrito?agregar={id}` y Carrito agrega al cargar. **`/` sigue redirigiendo a `/catalogo`.** La sección 9 pasa de “Lo que falta acordar” a “Decisiones de la v1.2”. Plazo para objetar: 8 de noviembre de 2026